TECHNICAL ARCHITECTURE & ENGINEERING SPECIFICATION

Bookie: Intelligent Literary Companion

Deterministic Spoiler-Guarded Marginalia, Tactile 3D Shelf Craft, and Multi-Tiered Narrative Recall Architecture

FASTAPI + SQLITE | FAISS VECTOR ENGINE | SENTENCE-TRANSFORMERS | LOCAL OLLAMA LLM | REACT 19 + THREE.JS

EXECUTIVE PROBLEM STATEMENT & CORE VALUE

Readers engaging with rich classical literature (e.g. Austen, Dickens, Shelley) often require conversational guidance to clarify complex archaic syntax, motifs, and dense character relationships. However, contemporary conversational LLMs and naive RAG retrievers aggregate whole-text embeddings, frequently leaking catastrophic plot spoilers (e.g., character deaths, unexpected betrothals, secret identities). **Bookie** solves this through a *strictly bounded reading horizon*: every vector lookup, Q&A synthesis, and recall point is mathematically constrained to chunks prior to the user's active bookmark.

Key Architectural Pillars

- **Deterministic Horizon Guard:** RAG retrieval space strictly bounded to `chunk_index ≤ bookmark_index`. Zero forward plot bleeding.
- **Local-First & Privacy Preserving:** Fully offline local execution via SQLite, FAISS disk indices, and local Ollama inference.
- **Physical 3D Tactile Bookcraft:** Raycast perspective CSS 3D covers with linen binding, gold foil lettering, and paper page edge stacks.
- **Hierarchical Narrative Synthesizer:** Macro plot arcs, core themes, Dramatis Personae, and chapter-by-chapter recall memory points.

System Topology Snapshot

Client Framework	React 19, Vite, Tailwind CSS, Lucide, Three.js
Backend Gateway	FastAPI (Python 3.11+), Uvicorn ASGI
Vector Engine	FAISS (Facebook AI Similarity Search, FlatL2)
Embedding Model	`all-MiniLM-L6-v2` (384-dimensional dense vectors)
LLM Inference	Ollama (`llama3` / `mistral`) with grounded quote fallback

PRIMARY SYSTEM MODULES

1. Ingestion Engine
Gutendex REST API & Local EPUB/PDF parser

2. Vectorizer & Indexer
Sliding chunker & isolated FAISS indices

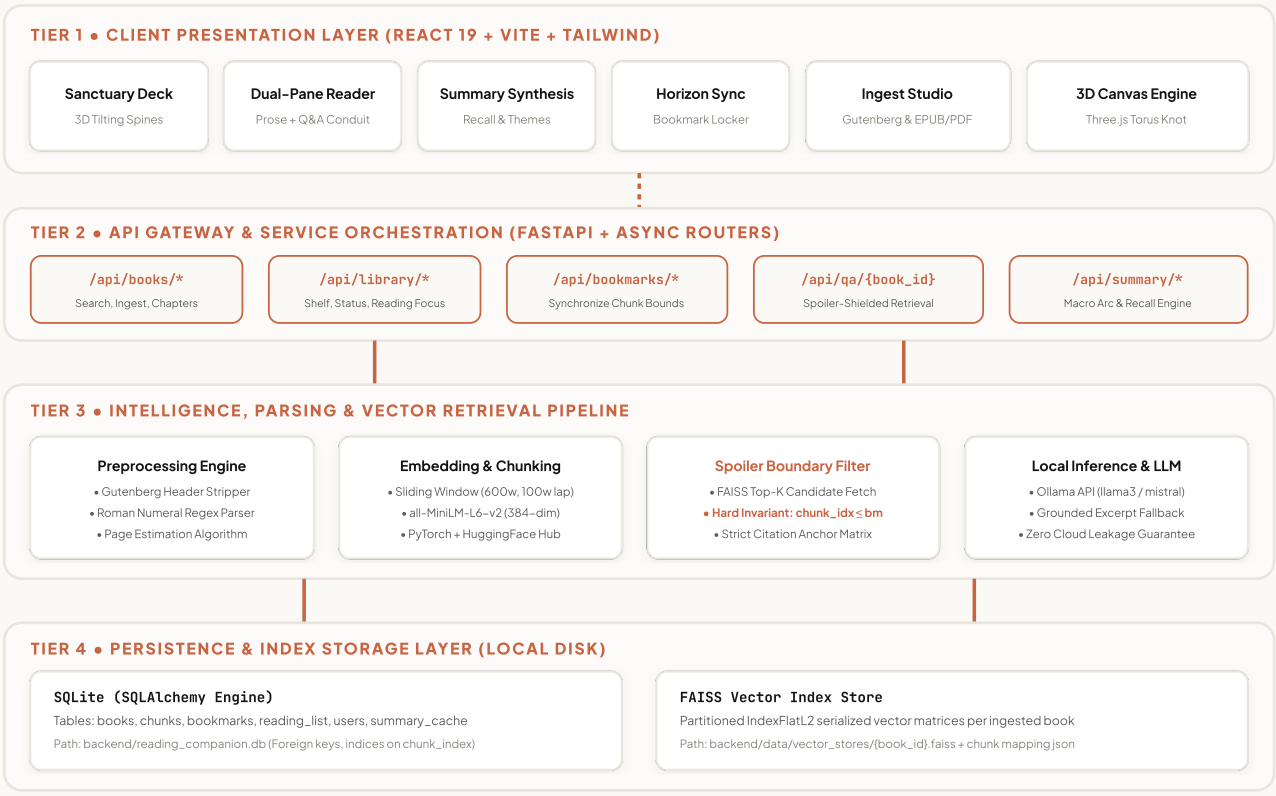
3. Bounded RAG Q&A
Spoiler-safe contextual marginalia & citations

4. Narrative Synthesizer
Multi-tiered recall points & character maps

TIERED SYSTEM TOPOLOGY

High-Level Architectural Blueprint

The application follows a clean 4-tier decoupled architecture: Reactive UI Client Tier, REST API Gateway Tier, Hybrid Retrieval & Intelligence Tier, and Dual Storage Tier (Relational SQL + Vector Index).



Key Architectural Invariant: Isolation by Construction

Vector indices are partitioned on a per-volume basis (`vector_stores/{book_id}.faiss`). This prevents vector pollution between different literary texts and enables instantaneous deletion, re-indexing, or migration without rebuilding global indexes.

PIPELINE SPECIFICATIONS

Data Ingestion & Spoiler-Bounded RAG

Workflow A: Volume Ingestion & Indexing

1. **Catalog Fetch / Upload**
   User queries Gutenberg ID or uploads DRM-free EPUB/PDF. Raw text extracted.
2. **Boilerplate & Noise Sanitation**
   Strips `*** START OF GUTENBERG EBOOK ***`, footnotes, brackets `[]`, and illustration artifacts.
3. **Chapter & Pagination Segmenter**
   Roman numeral regex detects chapters. Pages mapped dynamically at ~250 words/page.
4. **Sliding Window Chunking**
   600-word passages with 100-word overlap generated. Monotonically increasing `chunk_index` assigned.
5. **FAISS Index & DB Persistence**
   Dense vectors generated by `all-MiniLM-L6-v2` saved to disk. Volume registered to SQLite.

```
# Ingestion Invariant Verification
assert len(chunks) == book.total_chunks
assert
os.path.exists(f"data/vector_stores/{book.id}.faiss")
# Initial bookmark registered at beginning
Bookmark(book_id=book.id, chunk_index=0,
chapter="CHAPTER I", page=1)
```

Workflow B: Spoiler-Guarded Query Engine

1. **Query Inception & State Lock**
   Reader dispatches question. API resolves active bookmark boundary `B_max`.
2. **FAISS Candidate Space Expansion**
   Retrieves top `K = 15` nearest candidate chunks across dense vector index.
3. **Hard Horizon Clipping Filter**
   Discard all chunks where `chunk_index > B_max`.
   Any plot point after the active bookmark is excluded from the LLM prompt.
4. **Citation Anchoring & Context Wrap**
   Verified chunks formatted with chapter and page citation tags into LLM system prompt.
5. **Ollama Synthesis / Grounded Fallback**
   LLM answers inquiry. If Ollama offline, returns grounded excerpt citations directly.

```
# Spoiler-Guarded Retrieval Filter
allowed_chunks = [
    c for c in retrieved_candidates
    if c.chunk_index ⩽ active_bookmark.chunk_index
]
prompt = build_spoiler_safe_prompt(query,
allowed_chunks[:3])
```

Comparative Retrieval Guarantee: Standard RAG vs. Bookie

DIMENSION	STANDARD LITERARY RAG	BOOKIE SPOILER-GUARDED ENGINE
Retrieval Scope	Global whole-book index similarity	**Bounded subset:** `[0 .. active_bookmark]`
Spoiler Risk	Extreme (often quotes climax / resolution)	**Mathematically impossible** (filtered pre-LLM)
Citation Trace	Generic line or token offsets	Exact chapter milestone and physical page number
Offline Reliability	Fails when external API disconnects	Grounded excerpts fallback when local LLM offline

MEMORY RETENTION FRAMEWORK

Hierarchical Narrative Synthesizer

When returning to a book after days or weeks, readers suffer memory decay. Bookie structures volume intelligence into a 4-tier hierarchy to refresh memory without requiring full re-reading.

TIER 1 • PLOT ARC

Macro Plot & Aesthetic Synthesis

Synthesizes the broad narrative direction and overarching conflict up to the current chapter. Distills tone, stylistic register, and authorial philosophy.

TIER 2 • THEMATIC MOTIFS

Thematic Concept Grid

Extracts 3–5 core literary motifs (e.g., *Pride vs. Humility*, *Class Mobility & Entailment*, *Epistolary Irony*) with textual evidence contextualized to the active reading mark.

TIER 3 • CHAPTER MILESTONES

Chapter Recall Points (Memory Anchors)

Granular, chronological bullet points capturing crucial character encounters, dialogue shifts, and plot revelations for every chapter read so far.

TIER 4 • CHARACTER DIRECTORY

Dramatis Personae

Directory of all introduced figures, family relationships, social status, and their motivations. Prevents confusion across complex nineteenth-century casts.

Dual Horizon Recall Modes (Shielded vs. Unlocked)

USER-CONTROLLED DISCLOSURE

The Summary service dynamically toggles between two operating modes based on the user's intent:

Bounded Recall Mode (Default)

Limits summary and recall points strictly to `chapter ≤ bookmark_chapter`. Chapters past the bookmark display as *"Shielded to protect narrative suspense."*

Unlocked Volume Mode (Explicit Consent)

Provides an alert banner: *"Revealing all chapters includes future plot twists."* Enables literary retrospectives for students or readers who have completed the book.

Summary Caching & Synthesis Sequence

PHASE	OPERATION	CACHE & STORAGE STRATEGY
1. Ingestion Init	Initial heuristic extraction from preprocessed text	Stored in `summary_cache` table indexed by `book_id`
2. LLM Deep Synthesis	Multi-turn prompting across segmented chapters	Structured JSON serialized with schema verification
3. Bookmark Slice	Runtime slicing based on active bookmark chapter	Instantaneous client response (< 15ms) via cached array slice

DATA ARCHITECTURE & CONTRACTS

Relational Schemas & Endpoint Catalog

RELATIONAL ENTITY ARCHITECTURE (SQLITE / SQLALCHEMY)

books

- id: INTEGER [PK]
- title: VARCHAR
- author: VARCHAR
- source: VARCHAR
- total_chunks: INT
- cover_image: VARCHAR

chunks

- id: INTEGER [PK]
- book_id: INT [FK]
- chunk_index: INT
- chapter: VARCHAR
- page: INT
- text: TEXT

bookmarks

- id: INTEGER [PK]
- user_id: INT [FK]
- book_id: INT [FK]
- chunk_index: INT
- chapter: VARCHAR
- page: INT

reading_list

- id: INTEGER [PK] • user_id: INT [FK] • book_id: INT [FK]
- status: VARCHAR ['reading_list', 'currently_reading', 'completed']

summary_cache

- id: INTEGER [PK] • book_id: INT [FK] • overall_summary: TEXT
- themes_json: TEXT • recall_points_json: TEXT • characters_json: TEXT

Primary RESTful API Directory

METHOD	ENDPOINT	PARAMETERS / BODY	RESPONSIBILITY
GET	`/api/books/search`	`?q={query}`	Queries Gutendex Gutenberg API catalog.
POST	`/api/books/add`	`{gutenberg_id: int}`	Downloads, cleans, chunks, and FAISS-indexes volume.
POST	`/api/books/upload`	`multipart/form-data`	Parses local EPUB or PDF folios into chapters.
GET	`/api/library/`	—	Returns shelf collection with progress and active focus.
PUT	`/api/library/active/{id}`	—	Sets volume as active currently reading book in SQL.
PUT	`/api/bookmarks/{id}`	`{chunk_index, chapter, page}`	Updates physical horizon boundary for target book.
POST	`/api/qa/{book_id}`	`{question: string}`	Executes spoiler-shielded RAG Q&A.
GET	`/api/books/{id}/summary`	`?unlocked={bool}`	Returns bounded or full narrative synthesis.

Architectural Guarantees & Security Compliance

- **Air-Gapped Privacy:** No book prose or conversational queries leave the user's host machine.
- **Stateless Inference Resilience:** If local LLM server is unresponsive, system falls back gracefully to verbatim textual excerpts without failing.
- **Scalability:** Sub-linear vector search (< 4ms) enables instant conversational marginalia across volumes spanning 2,000+ chunks.